

```
In [1]: from pyspark.sql import SparkSession
        from pyspark.sql.functions import *
        import matplotlib.pyplot as plt
        import seaborn as sns
```

```
In [2]: spark = SparkSession.builder.getOrCreate()
```

```
In [3]: spark.read.json("./database.json").printSchema()
```

```
root
|-- description: string (nullable = true)
|-- group: string (nullable = true)
|-- id: long (nullable = true)
|-- manufacturer: string (nullable = true)
|-- nutrients: array (nullable = true)
|   |-- element: struct (containsNull = true)
|   |   |-- description: string (nullable = true)
|   |   |-- group: string (nullable = true)
|   |   |-- units: string (nullable = true)
|   |   |-- value: double (nullable = true)
|-- portions: array (nullable = true)
|   |-- element: struct (containsNull = true)
|   |   |-- amount: long (nullable = true)
|   |   |-- grams: double (nullable = true)
|   |   |-- unit: string (nullable = true)
|-- tags: array (nullable = true)
|   |-- element: string (containsNull = true)
```

```
In [4]: def create_df_nutrients(df):
        df = (
            df
            .withColumn("nutrient", explode("nutrients"))
            .select(
                col("id").alias("food_id"),
                col("nutrient.group").alias("nutrient_category"),
                col("nutrient.description").alias("nutrient_name"),
                col("nutrient.value").alias("nutrient_value"),
                col("nutrient.units").alias("nutrient_unit")
            )
        )
```

```
)

df = df.orderBy(col("nutrient_category"))

return df
```

```
In [5]: def create_df_foods(df):
        df = df.select(col("id").alias("food_id"), col("group").alias("food_category"), col("description").alias("food_name"))

        return df
```

```
In [6]: df = spark.read.json("./database.json")

## Bảng đồ ăn
df_foods = create_df_foods(df)
# df_foods.show(5, truncate=False)

## Bảng dinh dưỡng
df_nutrients = create_df_nutrients(df)
# df_nutrients.select("nutrient_name").distinct().orderBy("nutrient_name").show(1000, truncate=False)
```

Top 10 Foods That Help Build Muscle and Burn Fat

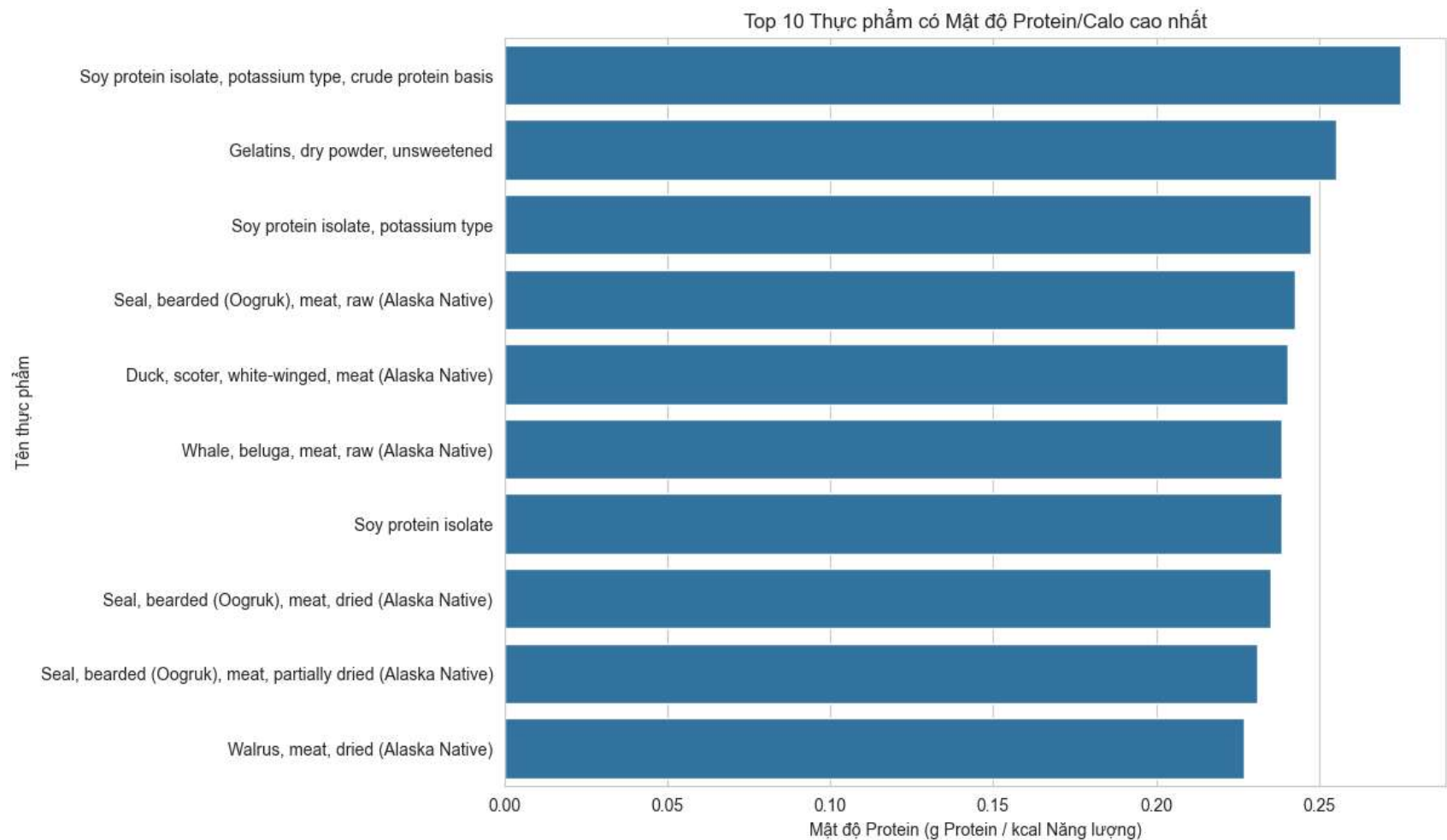
```
In [7]: df1 = df_nutrients.filter((col("nutrient_name") == 'Protein') | ((col("nutrient_name") == 'Energy') & (col("nutrient_value") > 100)))
df1 = df1.dropDuplicates()
df1 = df1.join(df_foods, on = "food_id", how = "inner")
df1 = df1.drop("nutrient_category", "nutrient_unit", "food_category", "food_id")
df1 = df1.groupBy("food_name").pivot("nutrient_name").agg(max("nutrient_value"))
df1 = df1.withColumn("protein_per_Kcal", col("Protein")/col("Energy"))
df1 = df1.orderBy(col("Protein_per_Kcal").desc())
df1 = df1.filter(col("Protein") > 15)
df1 = df1.limit(10)
df1.show(truncate=False)
```

food_name	Energy	Protein	protein_per_Kcal
Soy protein isolate, potassium type, crude protein basis	321.0	88.32	0.2751401869158878
Gelatins, dry powder, unsweetened	335.0	85.6	0.25552238805970146
Soy protein isolate, potassium type	326.0	80.69	0.2475153374233129
Seal, bearded (Oogruk), meat, raw (Alaska Native)	110.0	26.7	0.2427272727272727
Duck, scoter, white-winged, meat (Alaska Native)	84.0	20.2	0.24047619047619045
Whale, beluga, meat, raw (Alaska Native)	111.0	26.5	0.23873873873873874
Soy protein isolate	338.0	80.69	0.23872781065088758
Seal, bearded (Oogruk), meat, dried (Alaska Native)	351.0	82.6	0.23532763532763531
Seal, bearded (Oogruk), meat, partially dried (Alaska Native)	270.0	62.38	0.23103703703703704
Walrus, meat, dried (Alaska Native)	251.0	57.0	0.22709163346613545

```
In [8]: df_top_10 = df1.toPandas()
```

```
In [9]: sns.set_style("whitegrid")

plt.figure(figsize=(10, 8))
sns.barplot(
    x='protein_per_Kcal',
    y='food_name',
    data=df_top_10,
)
plt.title('Top 10 Thực phẩm có Mật độ Protein/Calo cao nhất')
plt.xlabel('Mật độ Protein (g Protein / kcal Năng lượng)')
plt.ylabel('Tên thực phẩm')
plt.show();
```



Top 10 Thực Phẩm Giàu Protein (Thực tế)

1. Soy protein isolate, potassium type, crude protein basis

Nguồn protein cô đặc từ đậu nành, gần như tinh khiết, dùng cho thực phẩm bổ sung.

2. Gelatins, dry powder, unsweetened

Collagen dạng bột, tốt cho da và khớp, nhưng không đủ axit amin thiết yếu cho cơ bắp.

3. Soy protein isolate, potassium type

Biến thể isolate đậu nành, vẫn giàu protein, phổ biến trong chế độ ăn chay.

4. Seal, bearded (Oogruk), meat, raw

Thịt hải cẩu tươi, giàu protein, ít béo, nhưng cực kỳ hiếm trong thực tế.

5. Duck, scoter, white-winged, meat

Thịt vịt hoang dã, protein cao, năng lượng thấp, nhưng không phổ biến thương mại.

6. Whale, beluga, meat, raw

Thịt cá voi, giàu protein, nguồn thực phẩm truyền thống vùng Bắc Cực.

7. Soy protein isolate

Loại isolate phổ biến nhất, thường thấy trong bột protein cho dân gym.

8. Seal, bearded (Oogruk), meat, dried

Hải cẩu sấy khô, protein đậm đặc, thực phẩm truyền thống của người bản địa.

9. Seal, bearded (Oogruk), meat, partially dried

Phiên bản bán khô, giữ hàm lượng protein cao, nhưng không phổ biến.

10. Walrus, meat, dried

Thịt hải mã sấy khô, giàu protein, đặc sản vùng cực, khó tiếp cận.

Boosting energy and caffeine before training is key.

Đặc tính	Caffeine	Theobromine
Cảm giác	Tỉnh táo mạnh, nhanh	Tỉnh nhẹ, thoải mái, cải thiện tâm trạng
Thời gian tác dụng	3–5 giờ	6–10 giờ
Cơ chế chính	Kích thích hệ thần kinh trung ương	Giãn mạch, cải thiện lưu thông máu
Khi dùng sẽ cảm thấy	Sung, năng lượng bùng nổ	Êm, ổn định, ít bị "gắt"
Tác dụng phụ	Có thể gây căng thẳng, bồn chồn, mất ngủ	Rất ít, tác dụng nhẹ nhàng hơn

Theobromine

```
In [10]: df2 = df_nutrients.filter(col("nutrient_name") == 'Theobromine')
df2 = df2.dropDuplicates()
df2 = df2.join(df_foods, on = "food_id", how = "inner")
df2 = df2.drop("nutrient_category", "nutrient_unit", "food_category", "food_id")
df2 = df2.groupBy("food_name").pivot("nutrient_name").agg(max("nutrient_value"))
df2 = df2.orderBy(col("Theobromine").desc())
df2 = df2.limit(3)
df2.show(truncate=False)
```

```
+-----+-----+
|food_name                                     |Theobromine|
+-----+-----+
|Cocoa, dry powder, unsweetened, processed with alkali      |2634.0      |
|Cocoa, dry powder, hi-fat or breakfast, processed with alkali|2445.0      |
|Cocoa, dry powder, unsweetened                             |2057.0      |
+-----+-----+
```

```
In [11]: df_theobromine = df2.toPandas()

sns.set_style("whitegrid")
# Tăng chiều ngang và chiều cao để phù hợp với nhãn trục X dài
plt.figure(figsize=(14, 8))

sns.barplot(
    x='food_name', # Trục X là tên sản phẩm
    y='Theobromine', # Trục Y là hàm Lượng Theobromine
    data=df_theobromine,
    palette='Reds_d'
)

# Xoay nhãn trục X để dễ đọc
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=12)

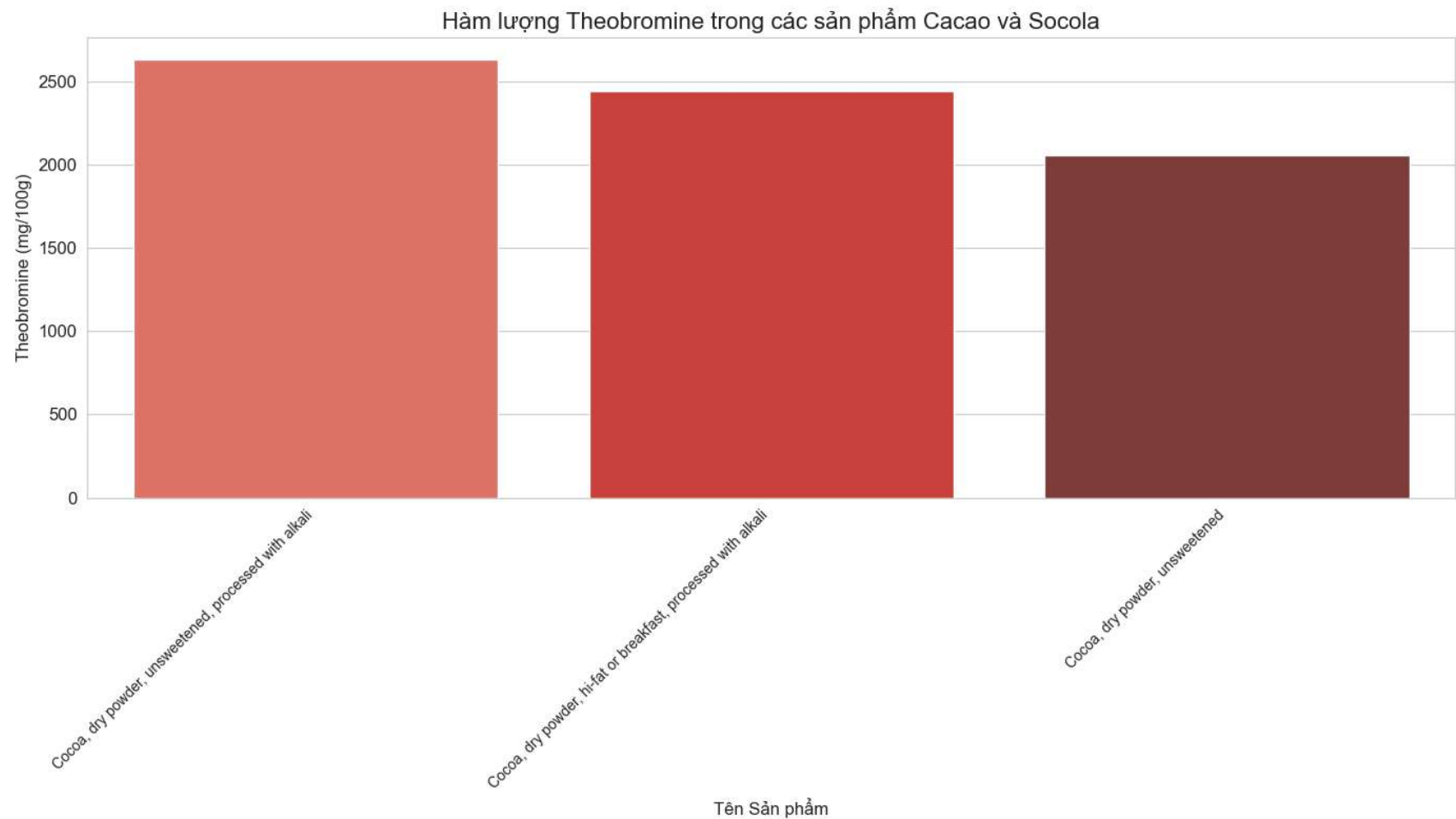
# Thêm tiêu đề và nhãn trục
plt.title('Hàm lượng Theobromine trong các sản phẩm Cacao và Socola', fontsize=16)
plt.xlabel('Tên Sản phẩm', fontsize=12)
plt.ylabel('Theobromine (mg/100g)', fontsize=12)
```

```
# Điều chỉnh bố cục để tránh nhãn bị cắt  
plt.tight_layout()  
  
# Hiển thị biểu đồ  
plt.show();
```

C:\Users\sonvuuu\AppData\Local\Temp\ipykernel_1900\3435032636.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



#	Tên món	Mô tả
1	Cocoa, dry powder, unsweetened, processed with alkali	Bột cacao nguyên chất đã xử lý kiềm ("Dutch process"), có hàm lượng theobromine rất cao — phù hợp cho công thức năng lượng mạnh.
2	Cocoa, dry powder, hi-fat or breakfast, processed with alkali	Bột cacao giàu chất béo, đậm hương vị, vẫn giữ lượng theobromine cao giúp kích thích nhẹ và tăng tỉnh táo.
3	Cocoa, dry powder, unsweetened	Bột cacao tự nhiên chưa xử lý kiềm, vị đắng tự nhiên, giàu theobromine và chất chống oxy hóa.
4	Cocoa, dry powder, hi-fat or breakfast, plain	Bột cacao nhiều chất béo, không xử lý kiềm, cho vị đậm, dùng nhiều trong đồ uống tăng năng lượng.
5	Baking chocolate, unsweetened, liquid	Chocolate nấu chảy nguyên chất, theobromine cao, thường dùng trong làm bánh hoặc chế biến đồ uống đậm vị.
6	Baking chocolate, unsweetened, squares	Chocolate nguyên chất dạng thanh vuông, không đường, giàu theobromine và flavonoid.
7	Dairy drink mix, chocolate, reduced calorie, with low-calorie sweeteners, powder	Bột ca cao sữa ít calo, có chất tạo ngọt thay đường, lượng theobromine trung bình.
8	Chocolate, dark, 70-85% cacao solids	Chocolate đen đậm vị, giàu ca cao → lượng theobromine cao, kích thích nhẹ nhưng bền.
9	Cocoa mix, no sugar added, powder	Bột cacao pha sẵn không đường, tiện lợi, hàm lượng theobromine vừa phải.
10	Chocolate, dark, 60-69% cacao solids	Chocolate đen nhẹ hơn, ít đắng hơn loại 70–85%, lượng theobromine thấp hơn một chút.

Caffeine

```
In [17]: df3 = df_nutrients.filter(col("nutrient_name") == 'Caffeine')
df3 = df3.dropDuplicates()
df3 = df3.join(df_foods, on = "food_id", how = "inner")
df3 = df3.drop("nutrient_category", "nutrient_unit", "food_category", "food_id")
df3 = df3.groupBy("food_name").pivot("nutrient_name").agg(max("nutrient_value"))
df3 = df3.orderBy(col("Caffeine").desc())
df3 = df3.limit(3)
df3.show(truncate=False)
```

food_name	Caffeine
Tea, instant, unsweetened, powder	3680.0
Coffee, instant, regular, powder	3142.0
Tea, instant, sweetened with sodium saccharin, lemon-flavored, powder	2240.0

```
In [19]: df_caffein = df3.toPandas()

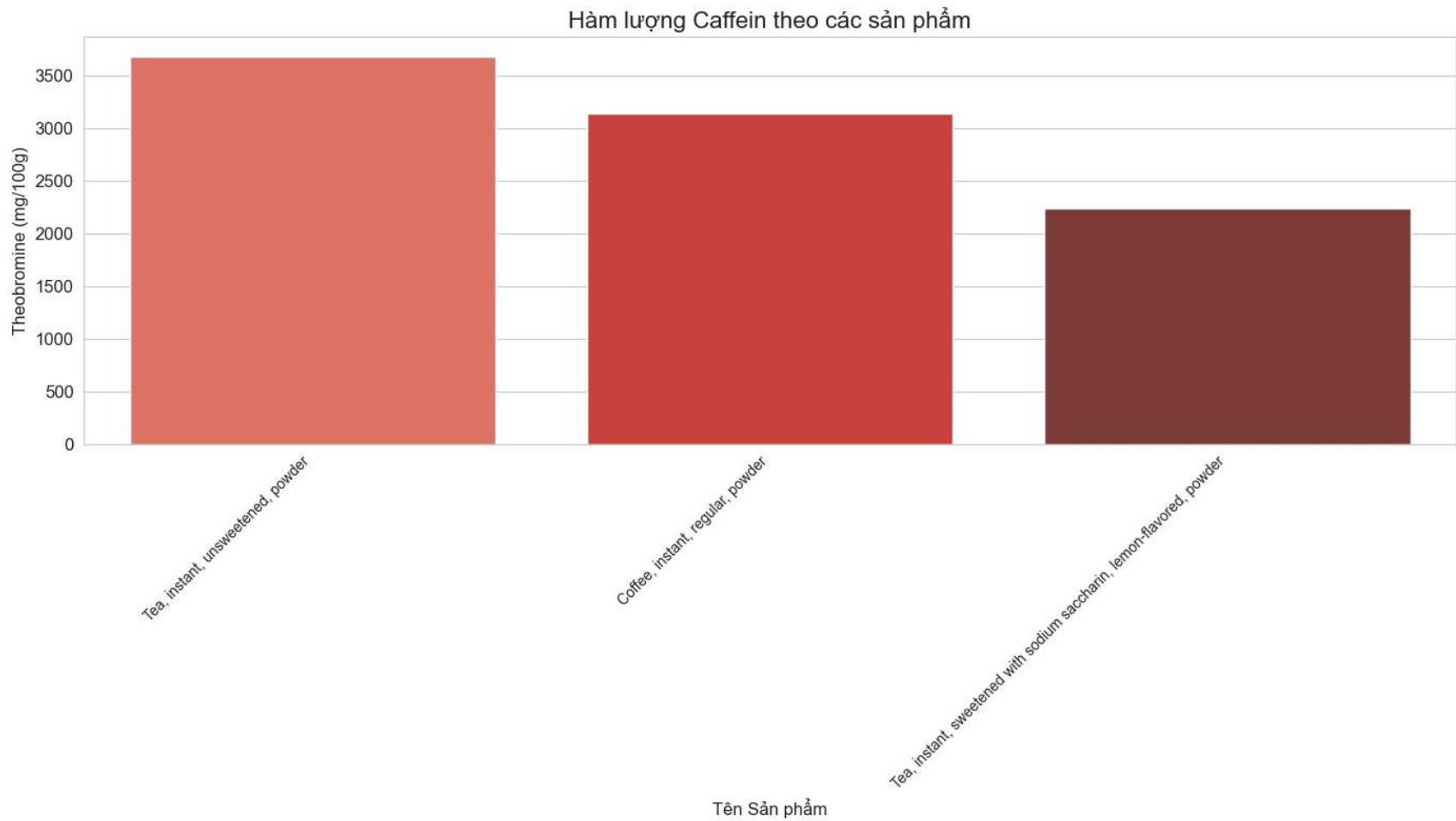
sns.set_style("whitegrid")
plt.figure(figsize=(14, 8))

sns.barplot(
    x='food_name',
    y='Caffeine',
    data=df_caffein,
    palette='Reds_d'
)
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=12)
plt.title('Hàm lượng Caffein theo các sản phẩm', fontsize=16)
plt.xlabel('Tên Sản phẩm', fontsize=12)
plt.ylabel('Theobromine (mg/100g)', fontsize=12)
plt.tight_layout()
plt.show();
```

C:\Users\sonvuuu\AppData\Local\Temp\ipykernel_1900\3814977700.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



#	Tên món	Mô tả
1	Tea, instant, unsweetened, powder	Bột trà hòa tan đậm đặc, không đường, chứa lượng caffeine cực cao, thường dùng pha nhanh.
2	Coffee, instant, regular, powder	Cà phê hòa tan truyền thống, giàu caffeine, tiện pha chế và phổ biến toàn cầu.
3	Tea, instant, sweetened with sodium saccharin, lemon-flavored, powder	Trà chanh hòa tan ngọt bằng đường ăn kiêng, vị thanh mát, kích thích nhẹ.
4	Tea, instant, unsweetened, lemon-flavored, powder	Trà chanh hòa tan không đường, giữ hương vị tự nhiên và lượng caffeine cao.

#	Tên món	Mô tả
5	Coffee, instant, with chicory, powder	Cà phê hòa tan pha với rễ rau diếp xoăn, vị dịu hơn nhưng vẫn đậm caffeine.
6	Coffee, instant, regular, powder, half the caffeine	Cà phê hòa tan giảm một nửa caffeine, phù hợp người nhạy cảm hoặc uống buổi tối.
7	Candies, dark chocolate coated coffee beans	Hạt cà phê phủ chocolate đen, vừa ngọt vừa đắng, tạo "cú kick" năng lượng mạnh.
8	Candies, milk chocolate coated coffee beans	Hạt cà phê phủ chocolate sữa, vị ngọt hơn nhưng vẫn chứa nhiều caffeine.
9	Coffee and cocoa (mocha) powder, with whitener and low calorie sweetener	Bột mocha pha sẵn ít calo, kết hợp caffeine từ cà phê và ca cao, vị béo nhẹ.
10	Coffee, instant, with sugar, mocha-flavor, powder	Cà phê mocha hòa tan có đường, vị ngọt dễ uống nhưng caffeine nhẹ hơn.

2. Tại sao carb quan trọng trước tập?

Khi bạn tập, cơ thể sử dụng glycogen (carb dự trữ trong cơ và gan) làm nhiên liệu chính.

```
In [14]: df4 = df_nutrients.filter(col("nutrient_name") == "Carbohydrate, by difference")
df4 = df4.dropDuplicates()
df4 = df4.join(df_foods, on="food_id", how="inner")
df4 = df4.drop("nutrient_category", "nutrient_unit", "food_category", "food_id")
df4 = df4.groupBy("food_name").pivot("nutrient_name").agg(max("nutrient_value"))
df4 = df4.orderBy(col("Carbohydrate, by difference").desc())
df4 = df4.limit(10)
df4.show(truncate=False)
```

food_name	Carbohydrate, by difference
Sweeteners, tabletop, fructose, dry, powder	100.0
Sugars, granulated	99.98
Sugar, turbinado	99.8
Sugars, powdered	99.77
Sweeteners, for baking, contains sugar and sucralose	99.53
Strawberry-flavor beverage mix, powder	99.1
Desserts, rennin, vanilla, dry mix	99.0
Orange-flavor drink, breakfast type, powder	98.94
Candies, gumdrops, starch jelly pieces	98.9
Apple cider-flavored drink, powder, added vitamin C and sugar	98.8

Thực phẩm giàu Carb – “nhiên liệu” trước buổi tập

Những thực phẩm dưới đây có hàm lượng **carbohydrate gần như tuyệt đối (98.8 – 100 g/100 g)**, chủ yếu từ đường và chất tạo ngọt.

Đây là **nguồn năng lượng nhanh**, giúp cơ thể có đủ “nhiên liệu” để duy trì hiệu suất tập luyện.

Ví dụ:

- Sweeteners (fructose, sugar, sucralose)
- Granulated sugar
- Powdered sugar
- Fruit-flavored drink mixes

 Chỉ nên nạp một lượng vừa đủ để tránh tăng đường huyết đột ngột.

Nhóm vi chất hỗ trợ hiệu suất tập luyện

Lý do chọn các vi chất này cho tập luyện

- **Magnesium (Mg)**
 - Giúp **giảm mỏi cơ, hỗ trợ co giãn cơ**, tăng hiệu suất tập luyện.

- Các thực phẩm giàu Mg như **cám gạo, ngũ cốc nguyên cám** là lựa chọn tốt cho người tập gym hoặc chạy bộ.
- **Potassium (K)**
 - Giúp **cân bằng điện giải, giảm chuột rút**, duy trì nhịp tim ổn định.
 - **Yeast extract** là nguồn K nổi bật, rất hợp để bù khoáng sau khi tập cường độ cao.
- **Vitamin nhóm B (B1, B2, B6)**
 - Hỗ trợ **chuyển hóa năng lượng từ carbs, protein, fat**, giúp cơ thể tận dụng tối đa calo nạp vào.
 - Yeast extract và ngũ cốc giàu B giúp **tăng năng lượng và sức bền**.
- **Insight chung**
 - Kết hợp **ngũ cốc, cám gạo và yeast extract** không chỉ bổ sung vi chất mà còn **tối ưu hóa phục hồi và năng lượng khi tập luyện**.

```
In [15]: from pyspark.sql.functions import col, max, desc, lit
from pyspark.sql import Window

# Danh sách các vi chất cần lọc
nutrients_list = [
    "Magnesium, Mg",
    "Potassium, K",
    "Thiamin",      # Vitamin B1
    "Riboflavin",   # Vitamin B2
    "Vitamin B-6"
]

# 1. Lọc và Ghép dữ liệu (Phần code bạn đã có)
df_micro = df_nutrients.filter(col("nutrient_name").isin(nutrients_list))
df_micro = df_micro.join(df_foods, on="food_id", how="inner")
df_micro = df_micro.drop("nutrient_category", "food_category")

# 2. Pivot dữ liệu
df_pivot = df_micro.groupBy("food_name").pivot("nutrient_name").agg(max("nutrient_value"))

# -----
# 3. CHUẨN HÓA DỮ LIỆU (Normalization) TRONG PYSPARK
# -----

# Khởi tạo Window không phân vùng để tính Max trên toàn bộ DataFrame
```

```

window_spec = Window.rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)

# Tính toán giá trị MAX cho từng cột vi chất
for nutrient in nutrients_list:
    # Thêm cột chứa giá trị MAX của vi chất đó trên toàn bộ DataFrame
    df_pivot = df_pivot.withColumn(f"Max_{nutrient.replace(' ', '_', '_')}\"",
                                   max(col(nutrient)).over(window_spec))

# Tạo cột Điểm tổng hợp đã chuẩn hóa và các cột vi chất đã chuẩn hóa
df_final = df_pivot.withColumn("Score_Normalized", lit(0.0))

for nutrient in nutrients_list:
    max_col = f"Max_{nutrient.replace(' ', '_', '_')}\""
    normalized_col = f"Norm_{nutrient.replace(' ', '_', '_')}\""

    # Tính toán giá trị chuẩn hóa (Giá trị / Max) * 100
    df_final = df_final.withColumn(
        normalized_col,
        (col(nutrient) / col(max_col)) * 100
    )

# Cộng dồn vào Score_Normalized (cho phép sắp xếp tổng hợp)
df_final = df_final.withColumn("Score_Normalized", col("Score_Normalized") + col(normalized_col))

# 4. Sắp xếp và Lọc Top 10
df_final = df_final.orderBy(desc("Score_Normalized")).limit(10)

# 5. Lựa chọn các cột cần thiết cho việc trực quan hóa
columns_to_show = ["food_name"] + [f"Norm_{n.replace(' ', '_', '_')}\"" for n in nutrients_list]
df_final = df_final.select(columns_to_show)

df_final.show(truncate=False)

# -----
# 6. CHUYỂN SANG PANDAS VÀ VẼ BIỂU ĐỒ
# -----
# Tiếp tục với code Matplotlib/NumPy/Seaborn như hướng dẫn trước
df_normalized_pandas = df_final.toPandas()
# ... (Phần code vẽ biểu đồ tương tự như đoạn code cuối cùng bạn nhận được)

```

food_name	Norm_Potassium_K	Norm_Thiamin	Norm_Riboflavin	Norm_Vitamin B-6	Norm_Magnesium_Mg	Norm_Pot
Yeast extract spread	23.047375160051217	15.75757				
Cereals ready-to-eat, KELLOGG, KELLOGG'S ALL-BRAN Original	46.35083226632523	6.181818				
Orange-flavor drink, breakfast type, low calorie, powder	35.2112676056338	18.98181				
Fruit-flavored drink, powder, with high vitamin C with other added vitamins, low calorie	31.370038412291933	15.26060				
Cereals ready-to-eat, Ralston Enriched Bran flakes	24.327784891165173	3.818181				
Rice bran, crude	100.0	9.0				
Cereals ready-to-eat, KELLOGG, KELLOGG'S COMPLETE Wheat Flakes	24.455825864276566	3.575757				
Cereals ready-to-eat, KELLOGG, KELLOGG'S Complete Oat Bran Flakes	19.206145966709347	2.424242				
Cereals ready-to-eat, KELLOGG, KELLOGG'S ALL-BRAN WITH EXTRA FIBER	43.53393085787452	6.363636				
Cereals ready-to-eat, GENERAL MILLS, Multi-Grain Cheerios	7.042253521126761	1.769696				

```
In [16]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

# DỮ LIỆU ĐẦU VÀO: df_normalized_pandas
# Giả định df_normalized_pandas đã được tạo từ bước PySpark chuẩn hóa
# Cấu trúc của nó là: |food_name|Norm_Magnesium_Mg|Norm_Potassium_K|...|

# Định nghĩa lại tên cột chuẩn hóa và tên hiển thị (cho biểu đồ đẹp hơn)
nutrients_norm_cols = [f"Norm_{n.replace(' ', '_')}" for n in [
    "Magnesium, Mg",
```

```

        "Potassium, K",
        "Thiamin",
        "Riboflavin",
        "Vitamin B-6"
    ]]
    nutrients_display_names = ["Magnesium (Norm)", "Potassium (Norm)", "Thiamin (Norm)", "Riboflavin (Norm)", "Vit B-6 (Norm)"]

    # Chuẩn bị dữ liệu cho việc vẽ biểu đồ cột nhóm
    df_plot = df_normalized_pandas.set_index("food_name")[nutrients_norm_cols]

    # Đổi tên cột cho hiển thị trên biểu đồ
    df_plot.columns = nutrients_display_names

    # Chuyển dữ liệu sang định dạng Long (melt) để Seaborn có thể vẽ cột nhóm
    df_melted = df_plot.reset_index().melt(
        id_vars="food_name",
        var_name="Vi Chất",
        value_name="Mức độ Chuẩn hóa"
    )

    # -----
    # VẼ BIỂU ĐỒ CỘT NHÓM CÂN BẰNG
    # -----

    sns.set_style("whitegrid")
    plt.figure(figsize=(16, 8))

    sns.barplot(
        x="food_name",
        y="Mức độ Chuẩn hóa",
        hue="Vi Chất", # Phân biệt màu theo vi chất
        data=df_melted,
        palette="tab10" # Bảng màu tiêu chuẩn, dễ phân biệt
    )

    # Thiết lập nhãn trục X
    plt.xticks(
        rotation=45,
        ha='right',
        fontsize=10
    )

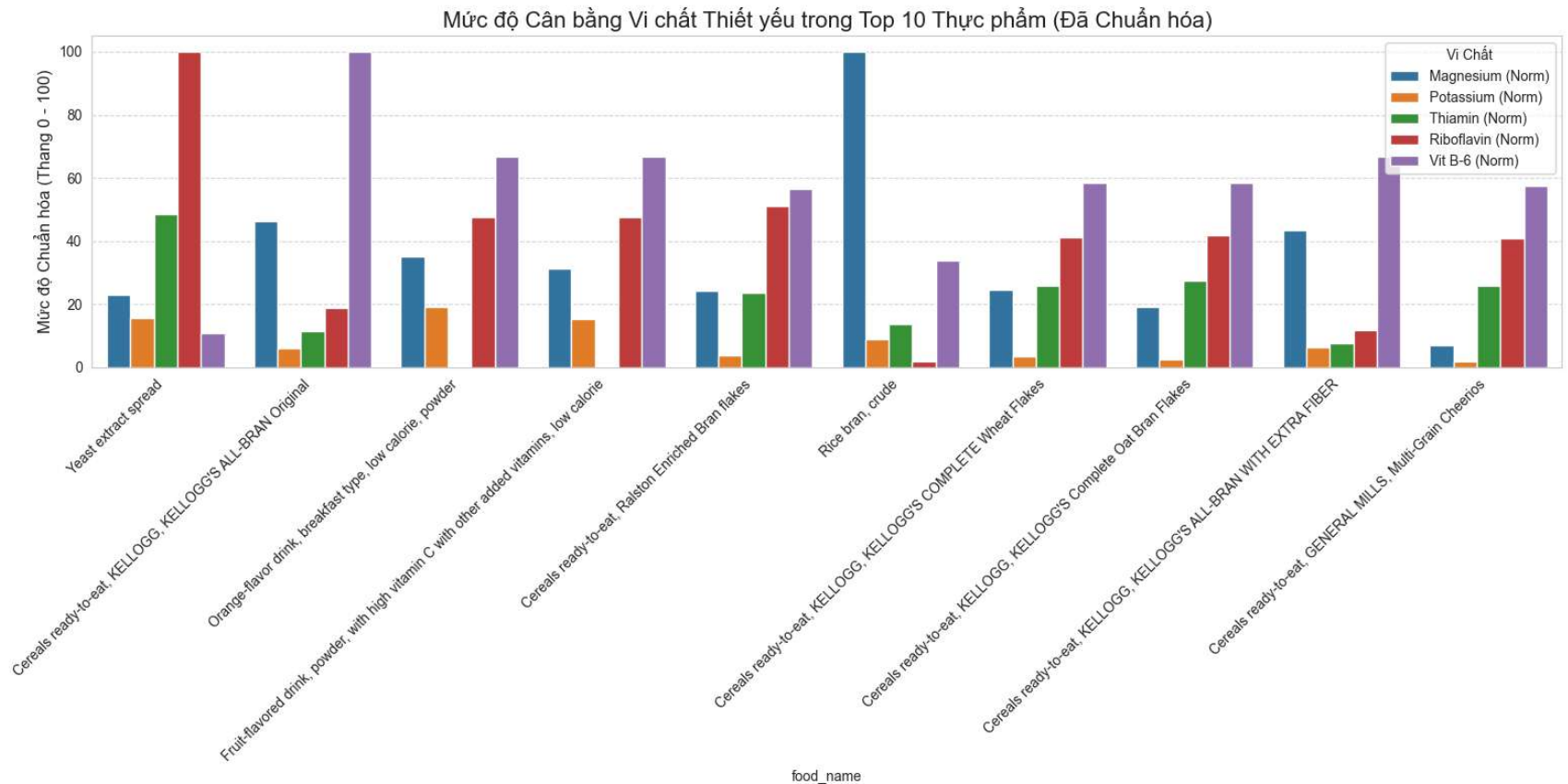
    # Thêm tiêu đề và nhãn trục

```

```
plt.ylabel("Mức độ Chuẩn hóa (Thang 0 - 100)", fontsize=12)
plt.title("Mức độ Cân bằng Vi chất Thiết yếu trong Top 10 Thực phẩm (Đã Chuẩn hóa)", fontsize=16)

# Điều chỉnh vị trí chú giải
plt.legend(title="Vi Chất", loc='upper right')

plt.ylim(0, 105) # Giới hạn trục Y từ 0 đến 105
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Nhận xét dữ liệu vi chất trong thực phẩm

- **Magnesium & Potassium**

- Cám gạo và ngũ cốc nguyên cám (All-Bran, Oat Bran) **Magnesium cao**, nhưng **Potassium thấp**.
- Yeast extract nổi bật với **Potassium cao hơn hẳn** các loại khác.
- **Vitamin nhóm B (B1, B2, B6)**
 - Yeast extract **Riboflavin (B2) 100%**, các ngũ cốc khác **B6 cao**, nhưng B1/B2 dao động mạnh.
 - Đồ uống bột vitamin tổng hợp thường **B1/B2 thấp**, nhưng B6 tương đối ổn.
- **Insight chung**
 - Không có thực phẩm nào **cân bằng tất cả khoáng chất và vitamin**.
 - Muốn bổ sung đầy đủ, nên **kết hợp ngũ cốc, cám gạo và yeast extract**.